COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	8/28/26	**DATE SUBMITTED:**	8/28/26
NAME:	SUBALISID, JAPPETH ROYCE B.	**INSTRUCTOR:**	ENGR. JIMLORD M. QUEJADO

Hands-on Activity 6.1 Searching Techniques

1. PROCEDURE OUTPUT

What procedures did you perform?

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

For each provided problem, give a screenshot of your code, the output console, and your answers to the Questions.

1. Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'?

Array implementation:

```
template <typename A>
int linearSearchAr(int size, A dataFind, A array[]) { //modified to return number of runs
    int i;
    for(i = 0; i < size; i++) {
        if (array[i] == dataFind){
            //std::cout << "Found Successfully in " << i+1 << "th position.\n";
            return i + 1; //return number of passes
        }
    }
    //std::cout << "Item not found.\n";
    return -1;
}
```

```
int arr1[10] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
int find = 18;
int n = linearSearchAr(10, find, arr1);
if(n > 0){
    std::cout << "It took " << n << " comparison/s to find " << find << std::endl;
} else {
    std::cout << "Item: " << find << ", not found.\n";
}
```

```
It took 2 comparison/s to find 18
```

Linked List implementation:

```cpp
template<typename A>
int linearSearchLL(Node<A> *nodes, A dataFind) {//modded to return passes
    int i = 1;
    while(nodes != nullptr) {
        if(nodes->data == dataFind){
            //std::cout << "The data: " << dataFind << ", was found successfully.\n";
            return i; //return passes
        }
        nodes = nodes->next;
        i++;
    }
    //std::cout << "Not found.\n";
    return -1;
}
```

```cpp
Node<int>* num1 = newNode(15);
Node<int>* num2 = newNode(18);
Node<int>* num3 = newNode(2);
Node<int>* num4 = newNode(19);
Node<int>* num5 = newNode(18);
Node<int>* num6 = newNode(0);
Node<int>* num7 = newNode(8);
Node<int>* num8 = newNode(14);
Node<int>* num9 = newNode(19);
Node<int>* num10 = newNode(14);

num1->next = num2;
num2->next = num3;
num3->next = num4;
num4->next = num5;
num5->next = num6;
num6->next = num7;
num7->next = num8;
num8->next = num9;
num9->next = num10;

int find = 18;
int n = linearSearchLL(num1, find);
if(n > 0){
    std::cout << "It took " << n << " comparison/s to find " << find << std::endl;
} else {
    std::cout << "Item: " << find << ", not found.\n";
}
```

```
It took 2 comparison/s to find 18
```

2. Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element ‘k’. Test on the same list given in problem 1.

```
template <typename A>
int linearSearchAr(int size, A dataFind, A array[]) { //modified to return number of copies
    int ins = 0;//no of isntance
    for(int i = 0; i < size; i++) {
        if (array[i] == dataFind){
            //std::cout << "Found Successfully in " << i+1 << "th position.\n";
            ins++; //add to instances
        }
    }
    //std::cout << "Item not found.\n";
    return ins;
}
```

```
int arr1[10] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
int find = 18;
int n = linearSearchAr(10, find, arr1);
std::cout << "The data: " << find << ", was found " << n << " times.\n";
```

```
The data: 18, was found 2 times.
```

3. Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

- Iteration 1

3	5	6	8	11 (Middle)	12	14	15	17	18

  - Since 8 < 11, algorithm gets rid of middle up to end

- Iteration 2

3	5	6 (Middle)	8

  - Since 8 > 6, algorithm gets rid of start to middle

- Iteration 3

8 (Middle)

  - Since it is the middle, and only element, the function prints ‘successfully found’

- Screenshot:

```
int arr2[10] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
int find = 8;
binarySearchAr(arr2, 10, find);
```

```
The data: 8, was found.
```

4. Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3

```
template<typename A>
void recBinarySearchAr(A data[], int low, int high, A dataFind) {//included high and low into parameters
    int middle = (low + high)/2;
    if (low > high){ //check if fail to return immediately
        std::cout << "The data: " << dataFind << ", was not found.\n";
        return;
    }
    if (data[middle] == dataFind){ //check if found to return immediately
        std::cout << "The data: " << dataFind << ", was found successfully.\n";
        return;
    } else if (dataFind < data[middle]) { //shrink size accordingly with function calls
        recBinarySearchAr(data, low, middle -1, dataFind);
    } else {
        recBinarySearchAr(data, middle +1, high, dataFind);
    }
}
```

```
int arr2[max_size] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
int find = 8;
recBinarySearchAr(arr2, 0, max_size - 1, find); //changed since were taking high and low
```

```
The data: 8, was found successfully.
```

Provide the following:

- Summary of lessons learned
  - In this lab activity, I have learned the two implementations of linear and binary search using linked lists and arrays. I also learned about when to use either since they require two different kinds of datasets; linear taking any data set and binary taking a sorted set. Though linear search is more versatile since it can search through unsorted data, it takes longer for it to find values compared to binary search.
- Analysis of the procedure
  - In the procedure, our professor did most of the work but that's where we learned how to do the custom header file for the two searching algorithms. We created the linear search using the array and linked list first before moving on to the binary search using both implementations too.
- Analysis of the supplementary activity
  - The supplementary activity was a little easy at first since it just needed a little modification to count things—number of comparisons and number or copies. But when I moved on to the last problem, which needed recursion—a function calling itself inside the function—is when I had troubles. Since I wanted to finish it quickly, I thought I just needed to change the parameters a little bit and the operations in the if-else statements, but when I did that, I found myself in an infinite loop with the constant calling of the while whenever I called the function inside. I then realized after thinking through the whole day, that the recursion is the loop itself, so I need to check the fail condition first before going through the success and other conditions, without using a for/while loop. There I was able to finish the activity quickly.
- Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?
  - I think I did pretty well in completing the supplementary activity, and since I knew a little already from the assignment we did a few weeks back, I understood the logic of both types of searching

algorithms and was able to apply other techniques to complete the tasks. I'm still a beginner in terms of the whole coding stuff, so I think I have to improve in all aspects.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.